Action Forge AI

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements for the

Generative AI-II course project

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

Candidate Name

J. Pranav Krishna (AP23110010457)

R. Sai Nitish (AP23110010401)

D. Chaitanya Raju (AP23110010470)

D. Sandeep (AP23110010400)

A. Teja Manoj (AP23110010413)

Under the Guidance of

Rithvik Konakala

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

APRIL, 2026

Contents

1 Project Overview 5

2 Problem Statement 6

3 Objectives 7

4 Core Technologies Used 8

5 Key Features 9

5.1 Meeting Notes Input & Processing 9

5.2 Executive Summary Generation 9

5.3 Automatic Task Extraction 9

5.4 Deadline Detection & Urgency Classification 9

5.5 Role Assignment Identification 9

5.6 Follow-up Email Drafting 10

5.7 Session Memory System 10

5.8 Export & Analytics 10

5.9 Collaboration & Authentication 10

6 Target Users 11

7 Real-World Applications 12

7.0.1 Scenario 1: Weekly Sales Review 12

7.0.2 Scenario 2: Client Meeting Follow-up 12

7.0.3 Scenario 3: Cross-Meeting Context Tracking 12

7.0.4 Scenario 4: Audio Meeting Processing 12

8 TECHNICAL ARCHITECTURE 14

8.1 High-Level Architecture Diagram 16

8.2 Component Interaction Diagram 17

9 Pre-requisites 18

9.1 Software Requirements 18
9.2 Libraries / Frameworks / Dependencies 18
9.3 Hardware Requirements 19

10 Prior Knowledge Required **20**
10.1 Programming Language Knowledge 20
10.2 Framework Basics 20
10.3 AI / NLP Fundamentals 20
10.4 Frontend Basics 20

11 Project Objectives (Detailed) **22**
11.1 Technical Objectives 22
11.2 Performance Objectives 22
11.3 Deployment Objectives 22
11.4 Learning Outcomes 23

12 Project Workflow **24**
12.1 Milestone 1: Requirement Analysis & System Design 24
12.1.1 Activity 1.1: Problem Definition 24
12.1.2 Activity 1.2: Functional Requirements 24
12.1.3 Activity 1.3: Non-Functional Requirements 25
12.1.4 Activity 1.4: System Design & Technology Selection 25
12.2 Milestone 2: Environment Setup & Initial Configuration 26
12.2.1 Activity 2.1: Development Environment Setup 26
12.2.2 Activity 2.2: Dependency Installation 27
12.2.3 Activity 2.3: Project Structure Creation 28
12.2.4 Activity 2.4: Configuration Setup 28
12.3 Milestone 3: API Key Setup 29
12.3.1 Activity 3.1: Create Groq Account & Get API Key 29
12.3.2 Activity 3.2: Create .env Configuration File 30
12.4 Milestone 4: Core System Development 31

12.4.1 Activity 4.1: Prompt Templates Module (prompts.py) 31
12.4.2 Activity 4.2: Tool Pipeline Module (tools.py) 32
12.4.3 Activity 4.3: LLM Integration Module (llm.py) 33
12.4.4 Activity 4.4: FastAPI Application (main.py) 34
12.5 Milestone 5: Integration & Optimization 35
12.5.1 Activity 5.1: Component Integration (Frontend, Backend, AI) . . 35
12.5.2 Activity 5.2: Performance Optimization 36
12.5.3 Activity 5.3: Security Enhancements 36
12.5.4 Activity 5.4: Error Handling & Validation 36
12.6 Milestone 6: Deployment 37
12.7 Milestone 7: Testing & Validation 39
12.7.1 Activity 7.1: Functional Test Case Execution 39

13 Project Directory Structure **41**

14 Results **42**

15 Advantages & Limitations **48**
15.1 Advantages 48
15.2 Limitations 48

16 Future Enhancements **50**
16.1 Scalability Improvements 50
16.2 Feature Expansion 50
16.3 Cloud Integration 50
16.4 Mobile Integration 50
16.5 Automation 51

17 Conclusion **52**

18 Appendix **53**
18.1 Configuration Files 53

18.2 API Documentation 53
18.3 GitHub Repository Link 53

Project Overview

ActionForge AI is an intelligent meeting-to-strategy platform built to automate the conversion of unstructured sales meeting notes into structured, executable action plans. Sales managers conduct weekly review meetings and document notes manually — this system takes those raw notes as text input and processes them through a multi-step AI pipeline to produce meaningful outputs.

The platform continuously processes text data to extract tasks, detect deadlines, identify role assignments, and generate follow-up emails. It goes beyond simple note-taking by identifying hidden commitments, mapping responsibilities, and creating professional communication drafts.

An integrated AI module powered by Groq's Llama 3.3 70B model generates executive summaries and personalized follow-up emails, acting as a virtual meeting assistant that guides teams toward better accountability, faster execution, and streamlined communication.

Problem Statement

Sales managers regularly conduct **weekly review meetings** to track performance, discuss targets, and plan future strategies. During these meetings, important discussions, decisions, and action points are typically recorded as **unstructured notes**.

However, this manual approach creates several challenges:

- **Lack of structure:** Notes are written in free-form text, making it difficult to identify key tasks and decisions
- **Missed action items:** Important tasks may be overlooked or forgotten after the meeting
- **No clear ownership:** Responsibilities are not explicitly assigned to team members
- **Undefined deadlines:** Tasks often lack clear timelines, leading to delays in execution
- **Inefficient follow-ups:** Managers must manually draft emails and track progress, which is time-consuming
- **No continuity:** There is no system to connect insights across multiple meetings

Objectives

The primary objective of this project is to develop an AI-driven system that converts unstructured meeting notes into structured, actionable plans by intelligently analyzing text and providing data-driven outputs. The system aims to help sales teams extract key decisions, identify tasks, and assign deadlines automatically from raw meeting text.

Another key objective is to leverage a multi-step AI tool pipeline (MCP-style architecture) to sequentially process meeting notes through five specialized stages — summarization, task extraction, deadline detection, role assignment, and email generation. By transforming raw notes into actionable outputs, the platform enables proactive follow-up rather than reactive recall.

The project also focuses on building a session memory system that maintains context across multiple meetings, delivering continuity and smarter recommendations over time. This ensures that each new analysis benefits from prior meeting history, enabling the AI to track ongoing commitments and evolving priorities.

Additionally, the system aims to demonstrate the integration of modern technologies such as Python, FastAPI, LLM APIs, and Prompt Templates in creating a scalable, real-world intelligent application. It highlights how AI-powered pipelines and structured prompting can be effectively combined to solve practical team collaboration and productivity challenges.

Core Technologies Used

Technology	Purpose
Python	Core backend logic, data processing, and AI integration
FastAPI	High-performance API development and system communication
LLM APIs (Groq)	AI-powered analysis, summaries, and action plan generation using Llama 3.3 70B
Prompt Templates	Structured prompt engineering for each pipeline stage
Session Memory	In-memory context store for cross-meeting continuity

Key Features

Meeting Notes Input & Processing

- Accepts raw, unstructured meeting notes as text input
- Supports optional meeting date for accurate deadline calculation
- Audio file upload with automatic transcription via Whisper API
- Drag-and-drop audio upload interface

Executive Summary Generation

- Generates concise summaries capturing key discussion points
- Highlights decisions, action items, and strategic priorities
- Uses prior meeting context from session memory for continuity

Automatic Task Extraction

- Extracts individual action items from unstructured text
- Assigns priority levels (High / Medium / Low) to each task
- Identifies task owners from names mentioned in the notes

Deadline Detection & Urgency Classification

- Detects deadline mentions from contextual clues (e.g., “by Friday”, “next week”)
- Converts relative dates to absolute dates using the meeting date
- Classifies urgency levels: Immediate, Soon, or Flexible

Role Assignment Identification

- Maps extracted tasks to responsible team members

- Identifies roles and responsibilities from meeting context
- Displays assignments with color-coded avatar cards

Follow-up Email Drafting

- Automatically generates a professional follow-up email
- Includes summary, action items, deadlines, and assignments
- One-click copy-to-clipboard functionality

Session Memory System

- Stores context from previous meetings for continuity
- Visual memory indicator in navigation bar
- Clear memory option for fresh sessions

Export & Analytics

- PDF and CSV export functionality
- Analytics dashboard with Chart.js visualizations
- Priority/urgency distribution charts, top assignees, meeting timeline

Collaboration & Authentication

- User registration and login system
- Shared sessions with permission levels (View / Edit / Admin)

Target Users

- Sales managers conducting regular team meetings and client reviews
- Team leads who need to track action items and deadlines from meetings
- Project managers requiring structured follow-up documentation
- Business analysts converting discussions into actionable strategies
- Any professional who documents meeting notes and needs automated task extraction

Real-World Applications

This system bridges the gap between unstructured meeting discussions and structured, executable action plans. It supports team accountability, deadline management, and professional communication.

APPLICATION SCENARIOS / USE CASES:

Scenario 1: Weekly Sales Review

A sales manager pastes notes from a weekly team review meeting. The system extracts five action items including "finalize Q2 proposal" and "set up demo environment", assigns them to Ravi and Priya respectively, detects deadlines like "by end of week" and "next Tuesday", and drafts a follow-up email summarizing all commitments. The manager copies the email and sends to the team within seconds.

Scenario 2: Client Meeting Follow-up

After a client call, a business analyst inputs discussion notes containing deliverables, timeline commitments, and assigned responsibilities. ActionForge extracts all client-facing tasks with urgency classification, generates an executive summary, and creates a professional follow-up email ready to send to the client.

Scenario 3: Cross-Meeting Context Tracking

A project manager processes meeting notes over multiple weeks. The session memory retains context from previous meetings, enabling the AI to reference outstanding tasks ("Ravi's proposal from last week") and flag overdue items. The analytics dashboard shows task distribution trends over time.

Scenario 4: Audio Meeting Processing

A team lead records a standup meeting on their phone. They upload the audio file to ActionForge, which transcribes it using Whisper API and automatically processes the

transcription through the full AI pipeline, producing a complete action plan without manual note-taking.

TECHNICAL ARCHITECTURE

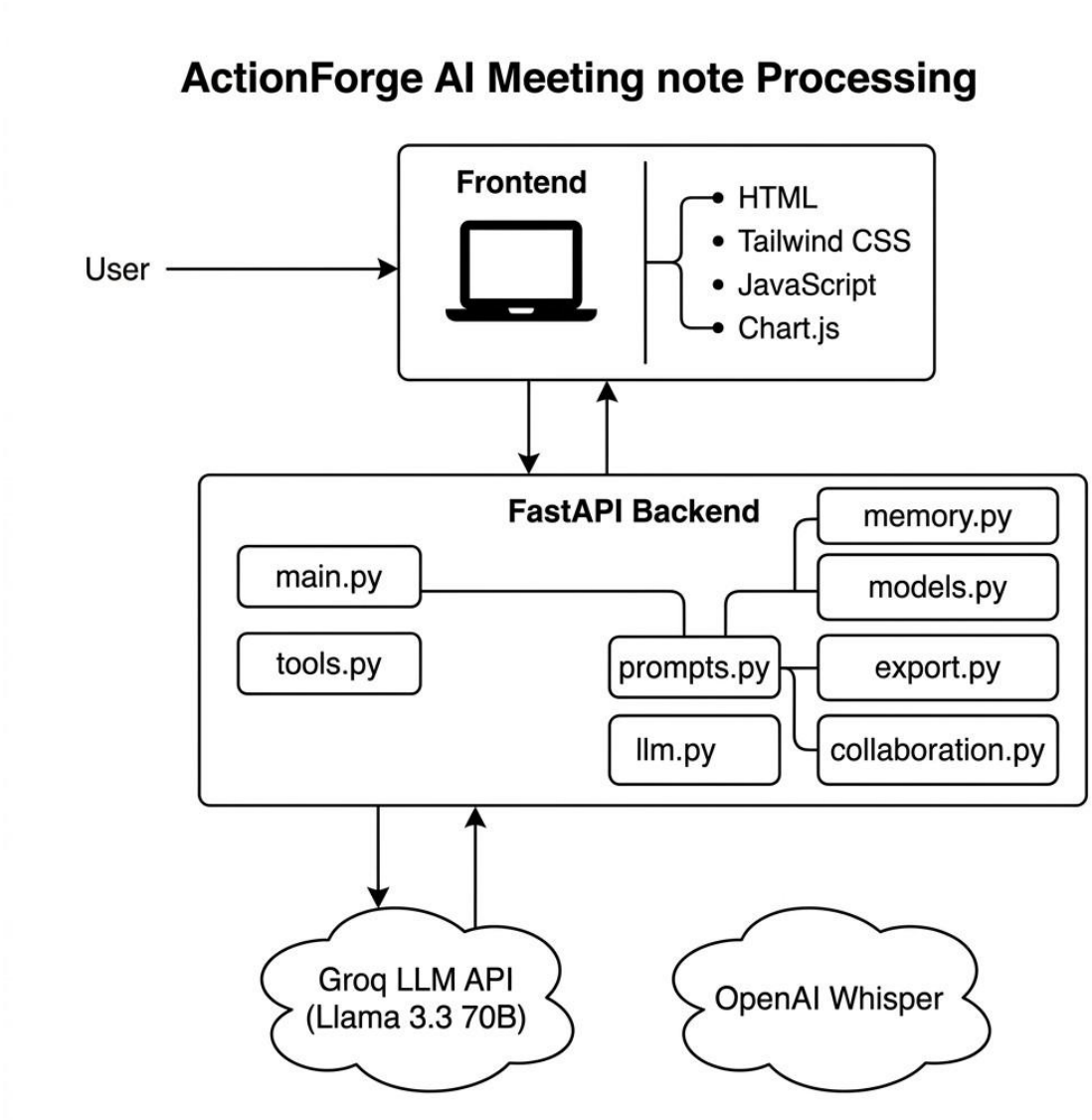

Figure 1: System Architecture — Frontend, FastAPI Backend, Groq LLM API, Session Memory flow

Frontend Layer

The frontend acts as the user interaction layer where users paste meeting notes, select dates, upload audio, and view generated action plans. It is a single-page HTML application styled with Tailwind CSS (CDN) and uses vanilla JavaScript for API communication and Chart.js for analytics visualizations.

Backend Layer

The backend is built using FastAPI and handles all core processing. It manages API requests, processes meeting notes through the 5-stage AI pipeline, maintains session memory, handles user authentication, manages collaboration sessions, and generates PDF/CSV exports.

AI Layer

The system integrates the Groq API with Llama 3.3 70B (primary) and Llama 3.1 8B (fallback) for all AI-powered text processing. Structured Prompt Templates guide the LLM through each analysis stage.

External APIs / Services

The system uses the Groq API for LLM-powered text analysis and OpenAI Whisper API for audio transcription. All API communication is handled securely through environment variables.

High-Level Architecture Diagram

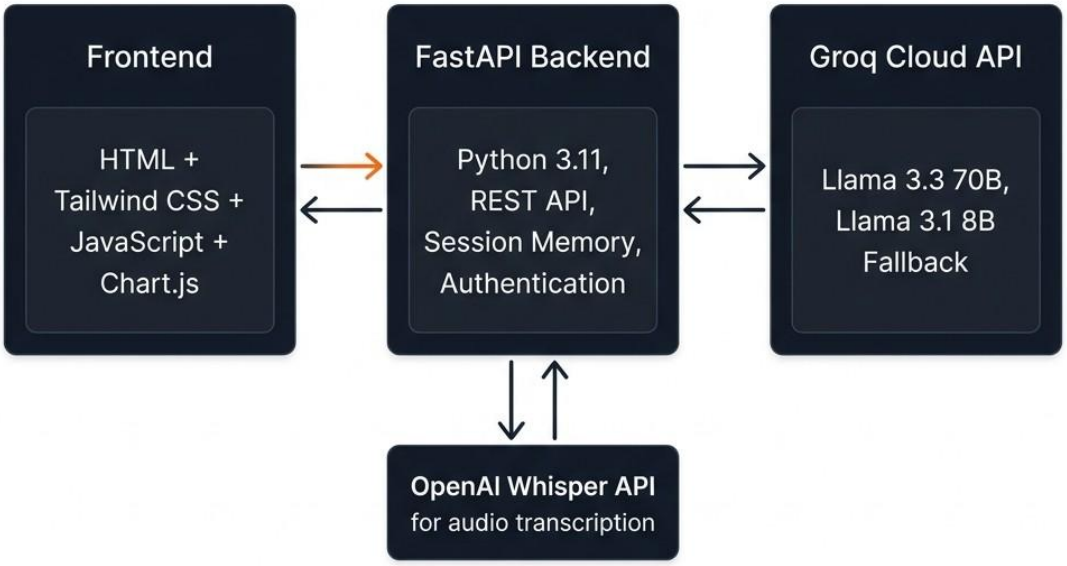

Figure 2: High-Level Architecture Diagram

Component Interaction Diagram

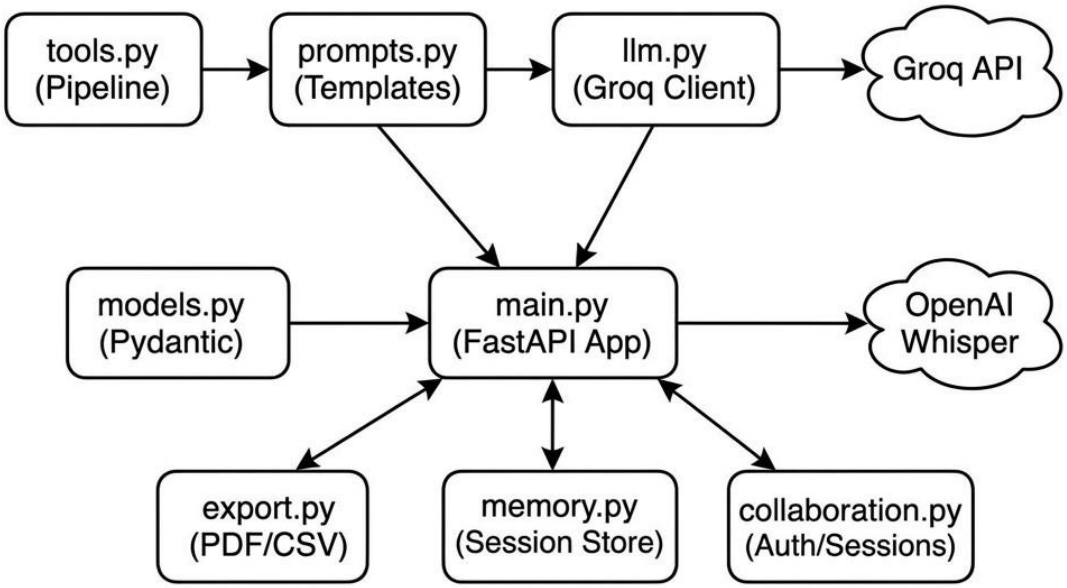

Figure 3: Component Interaction Diagram

Pre-requisites

Software Requirements

- **Python:** Version 3.9 or higher (Python 3.11 recommended)
- **pip:** Python package manager for installing project dependencies
- **IDE:** Visual Studio Code with Python extension or any preferred code editor
- **Browser:** Modern web browser (Chrome, Edge, Firefox) supporting ES6, async/await, and Fetch API
- **Git:** Version control system for project management (optional but recommended)

Libraries / Frameworks / Dependencies

Install required dependencies using:

```
pip install fastapi uvicorn groq pydantic python-dotenv reportlab
    python-multipart openai
```

- **FastAPI (0.115.0):** Web framework for REST API routing and request handling
- **Uvicorn (0.30.6):** ASGI server for running the FastAPI application
- **Groq (≥0.12.0):** Client library for accessing Groq LLM API
- **Pydantic (2.9.2):** Data validation and serialization models
- **python-dotenv (1.0.1):** Environment variable management from .env files
- **ReportLab (4.0.9):** PDF generation library for export functionality
- **python-multipart (0.0.6):** File upload handling for audio transcription
- **OpenAI (≥1.0.0):** Client library for Whisper audio transcription API

Hardware Requirements

- **CPU:** Minimum Intel Core i3 or AMD Ryzen 3 processor (AI processing is cloud-based via Groq)
- **RAM:** Minimum 4 GB (8 GB recommended for smooth development)
- **Storage:** At least 500 MB free disk space for project files and dependencies
- **Network:** Stable internet connection required for Groq API and OpenAI Whisper API calls

Prior Knowledge Required

Programming Language Knowledge

- Strong understanding of Python fundamentals (variables, loops, functions, modules)
- Knowledge of object-oriented programming (OOP) concepts
- Familiarity with handling JSON data and working with APIs
- Basic understanding of error handling and debugging techniques

Framework Basics

- Understanding of FastAPI framework for building RESTful APIs
- Knowledge of API routing, request/response handling, and status codes
- Familiarity with asynchronous programming (async/await) in Python
- Basic understanding of middleware (CORS) and dependency injection in FastAPI

AI / NLP Fundamentals

- Basic understanding of Large Language Models (LLMs) and their capabilities
- Knowledge of prompt engineering and structured prompt templates
- Familiarity with API-based AI services (Groq, OpenAI)
- Understanding of how text generation models process and produce outputs
- Awareness of prompt engineering basics for effective AI responses

Frontend Basics

- Understanding of HTML, CSS, and JavaScript fundamentals
- Familiarity with Fetch API for making HTTP requests

- Basic knowledge of DOM manipulation and event handling
- Understanding of responsive design and CSS frameworks (Tailwind CSS)

Project Objectives (Detailed)

Technical Objectives

- Design and develop a scalable meeting intelligence system using Python and FastAPI
- Implement a 5-stage AI tool pipeline for sequential meeting note processing
- Build structured Prompt Templates for each analysis stage
- Integrate Groq LLM API with automatic model fallback (Llama 3.3 70B → Llama 3.1 8B)
- Develop a session memory system for cross-meeting context retention
- Create RESTful API endpoints for all core functionalities

Performance Objectives

- Ensure fast API response times for real-time meeting note processing
- Maintain reliable LLM responses with dual-model fallback strategy
- Achieve accurate task extraction and deadline detection from unstructured text
- Provide reliable export generation (PDF/CSV) with minimal latency
- Optimize system for low latency in AI-generated outputs

Deployment Objectives

- Enable local development with simple setup (single pip install, single uvicorn command)
- Ensure secure handling of API keys using environment variables (.env files)
- Support cross-platform access via web browser
- Design for cloud deployment readiness

- Maintain high availability and reliability of backend services

Learning Outcomes

- Gain hands-on experience in building real-world AI-powered applications
- Understand integration of LLM APIs with structured prompt engineering
- Learn FastAPI development with Pydantic models
- Develop skills in API design, system architecture, and data flow management
- Improve knowledge of AI-driven text processing and NLP approaches

Project Workflow

Milestone 1: Requirement Analysis & System Design

Activity 1.1: Problem Definition

Identify the need for an intelligent meeting note processing system that not only records discussions but also extracts actionable outputs. Address the limitation of traditional note-taking tools that lack automatic task extraction, deadline detection, and follow-up generation. Design a solution that integrates text processing, AI-powered analysis, and structured output generation to help sales teams improve accountability, follow-through, and communication efficiency. Ensure the system is scalable, user-friendly, and capable of transforming raw meeting text into comprehensive action plans using LLM APIs and Prompt Templates.

Activity 1.2: Functional Requirements

- **FR-01:** Accept unstructured meeting notes as text input through a web interface
- **FR-02:** Support optional meeting date input for accurate deadline calculation
- **FR-03:** Generate executive summaries capturing key discussion points and decisions
- **FR-04:** Extract individual action items with priority classification (High/Medium/Low)
- **FR-05:** Detect deadlines from contextual clues and classify urgency (Immediate/-Soon/Flexible)
- **FR-06:** Identify and assign roles/responsibilities to team members mentioned in notes
- **FR-07:** Draft professional follow-up emails with all extracted information
- **FR-08:** Maintain session memory for cross-meeting context continuity

- **FR-09:** Export action plans as PDF and CSV documents
- **FR-10:** Transcribe audio meeting recordings to text using Whisper API
- **FR-11:** Expose RESTful API endpoints for all functionalities

Activity 1.3: Non-Functional Requirements

- **NFR-01 Performance:** API responses should complete within 10 seconds for full pipeline
- **NFR-02 Reliability:** Dual-model fallback ensures operation even if primary model fails
- **NFR-03 Scalability:** Modular architecture supports future feature additions
- **NFR-04 Security:** API keys managed through .env files, CORS configured
- **NFR-05 Maintainability:** Separation of concerns with distinct modules
- **NFR-06 Usability:** Clean, modern UI with intuitive input/output flow

Activity 1.4: System Design & Technology Selection

- **Backend Framework:** Use FastAPI for high-performance API development and async request handling
- **AI Integration:** Use Groq-powered LLMs (Llama 3.3 70B) for text analysis, summarization, and content generation
- **Prompt Engineering:** Design structured Prompt Templates for each of the 5 pipeline stages
- **Session Memory:** Implement in-memory context store for cross-meeting continuity
- **Frontend:** Single-page HTML with Tailwind CSS (CDN) and vanilla JavaScript
- **Export:** ReportLab for PDF, Python csv for CSV exports

- **Audio:** OpenAI Whisper API for speech-to-text transcription
- **Environment Configuration:** Use .env files to securely manage API keys

Milestone 2: Environment Setup & Initial Configuration

Activity 2.1: Development Environment Setup

- Install Python 3.11 from the official Python website and ensure "Add Python to PATH" is enabled during installation
- Install Visual Studio Code with the Python extension for development
- Create the main project directory: `mkdir actionforge && cd actionforge`
- Create backend and frontend directories: `mkdir backend frontend`
- Create and activate a virtual environment: `python -m venv venv`
- Activate: `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Linux/Mac)

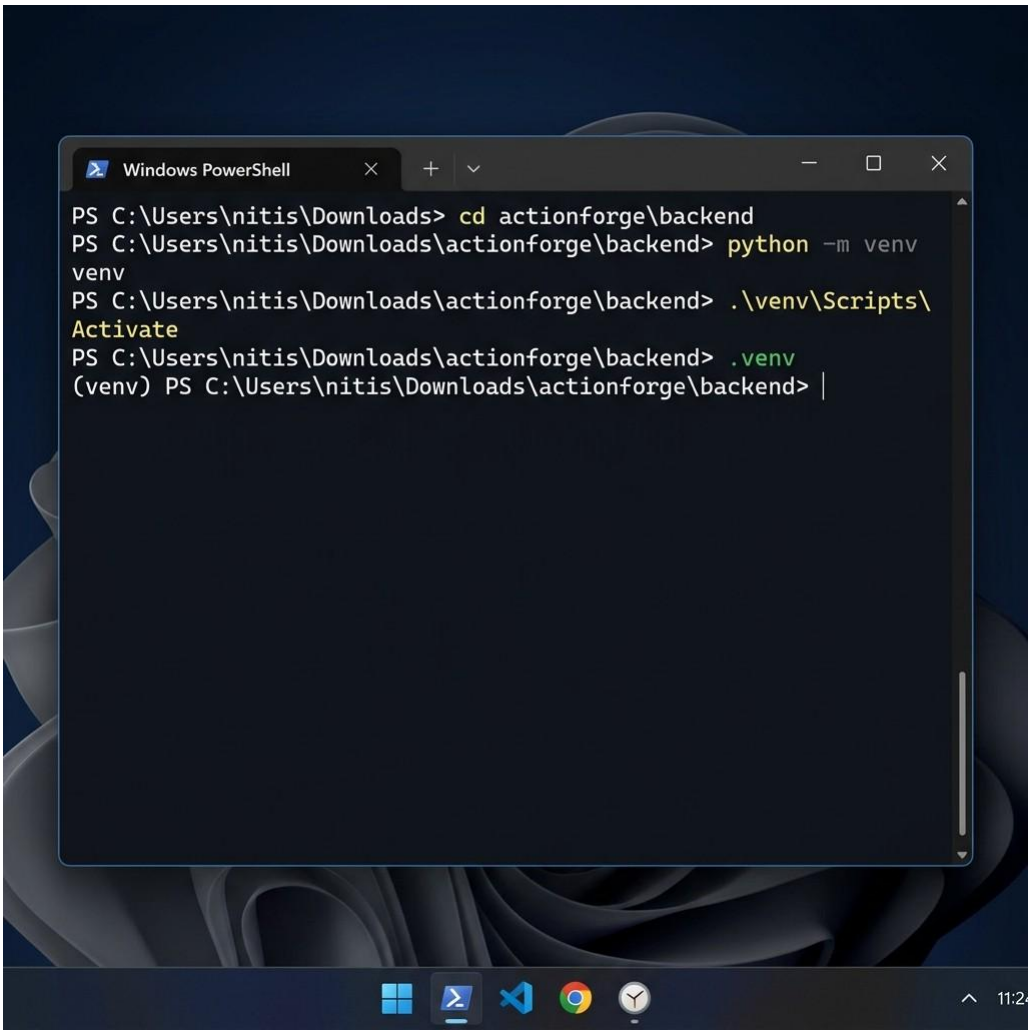

Figure 4: Virtual environment activated in terminal

Activity 2.2: Dependency Installation

Install all required Python libraries using pip:

```
pip install -r requirements.txt
```

requirements.txt contents:

```
fastapi==0.115.0
uvicorn==0.30.6
groq>=0.12.0
pydantic==2.9.2
python-dotenv==1.0.1
reportlab==4.0.9
```

```
python - multipart ==0 .0 .6
openai >=1 .0 .0
```

Activity 2.3: Project Structure Creation

Organize the project into a clean and modular folder structure by separating LLM logic, tools, prompts, models, memory, and export functions into individual files. This ensures better code readability, maintainability, and scalability while enabling smooth integration of different components.

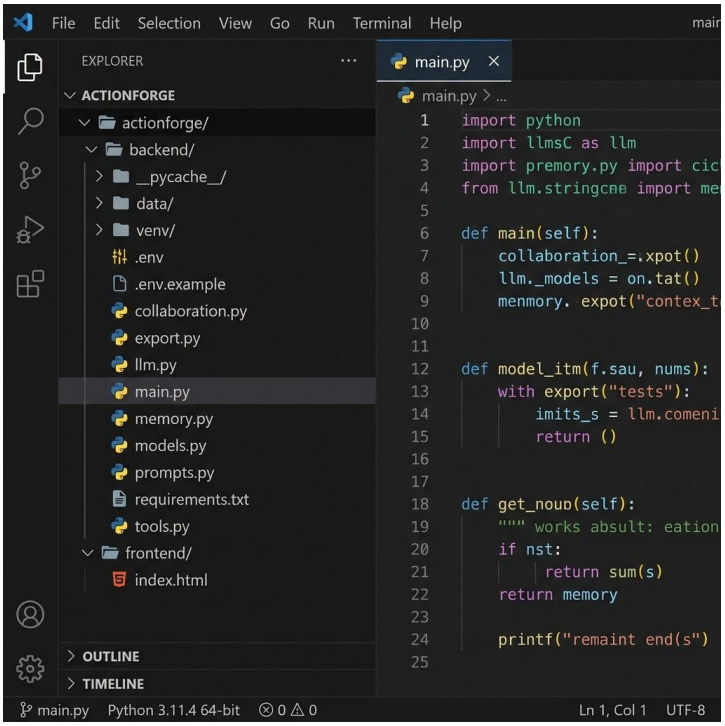

Figure 5: Project Folder Structure in VS Code

Activity 2.4: Configuration Setup

Set up the application configuration by defining environment variables required for secure and efficient operation. This includes storing API keys (Groq, OpenAI) in a .env file. Configure the LLM client initialization, model selection, and ensure all services are properly integrated.

Milestone 3: API Key Setup

Activity 3.1: Create Groq Account & Get API Key

Step 1: Create Groq Account

1. Go to Groq Console: https://console.groq.com
2. Click “Sign Up” or “Get Started”
3. Sign up with GitHub or Google (Free, no credit card required)
4. Verify your email if needed

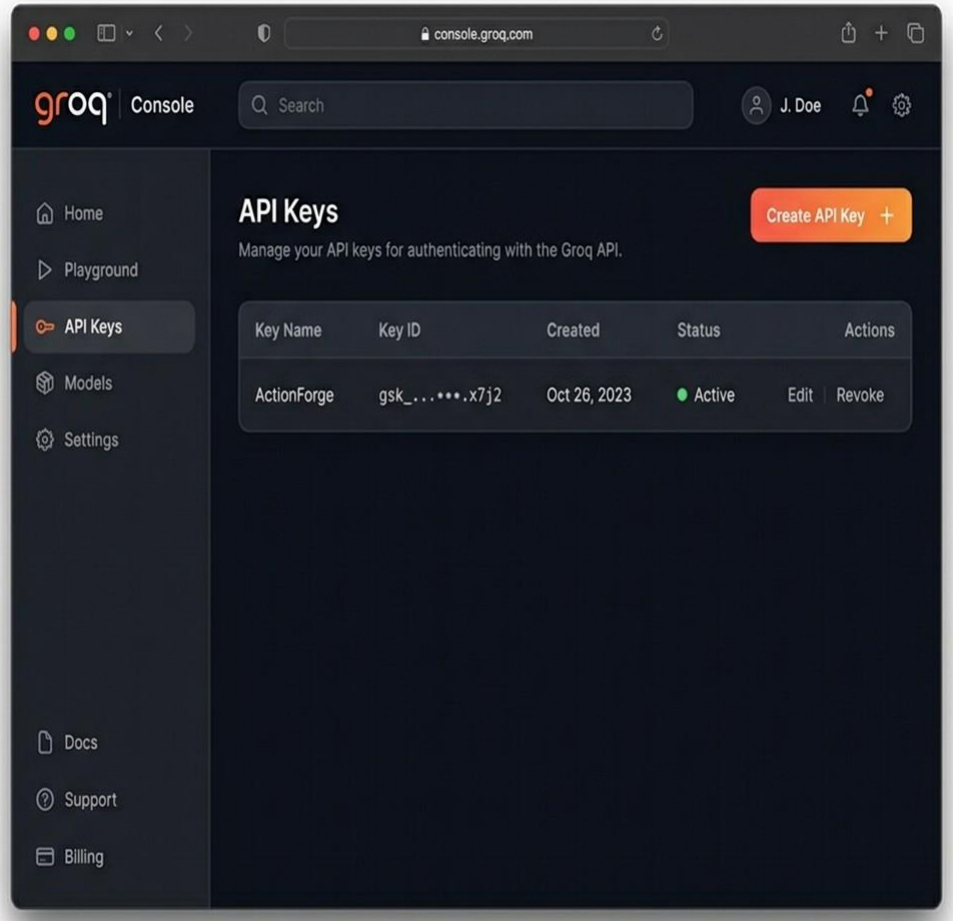

Figure 6: Groq Console Dashboard

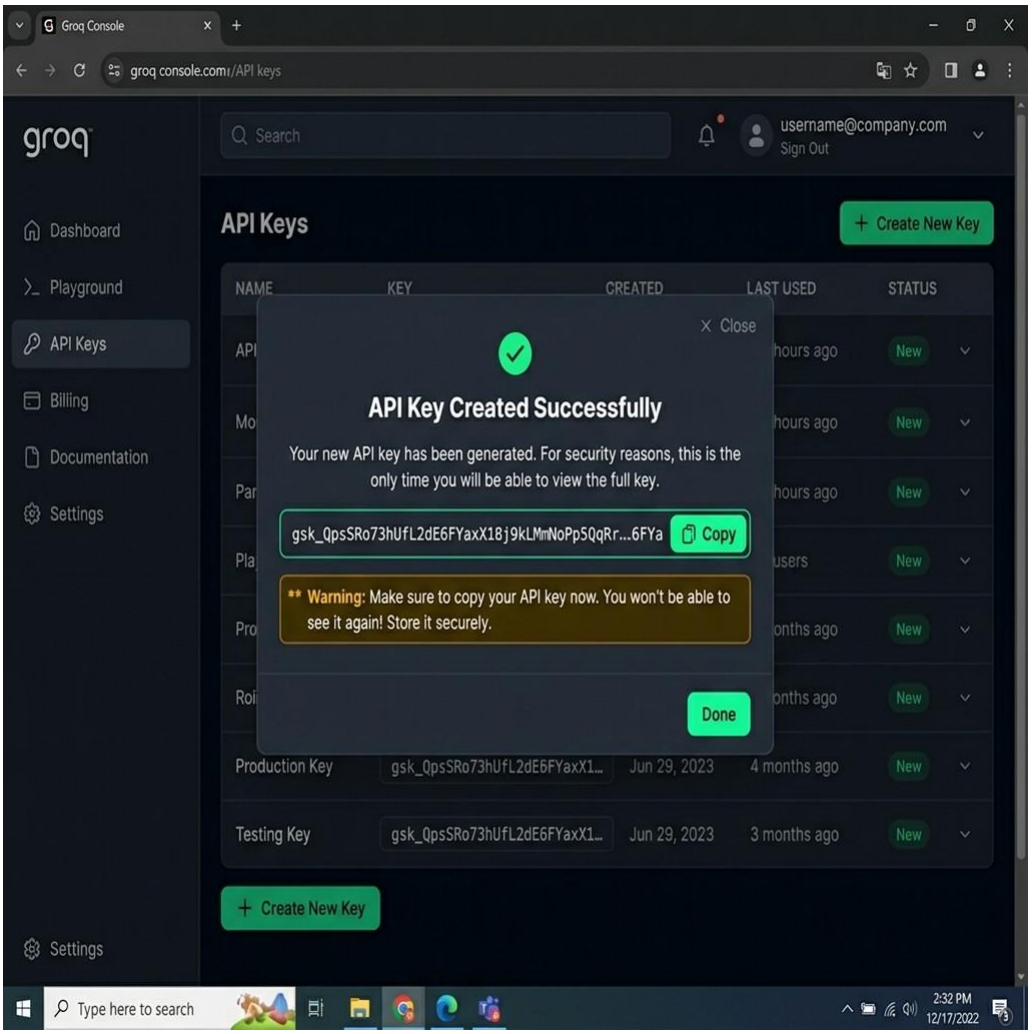

Figure 7: New API Key Created

Activity 3.2: Create .env Configuration File

1. Navigate to the backend/ directory

2. Create a new file named .env (not .env.txt)

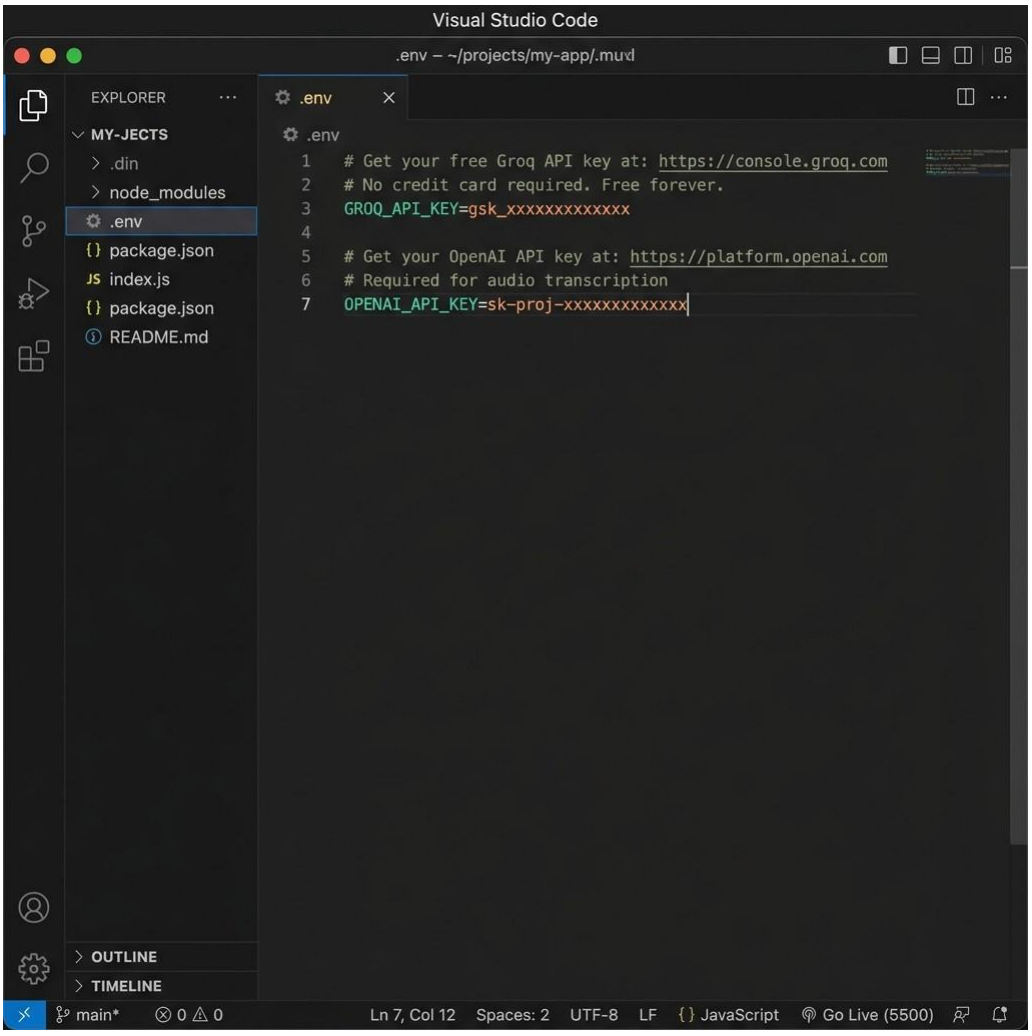

Figure 8: exttt.env File Configuration

Milestone 4: Core System Development

Activity 4.1: Prompt Templates Module (prompts.py)

This module defines structured system and user prompts for each of the 5 pipeline stages. Each prompt template is carefully engineered to guide the LLM to produce consistent, formatted outputs. The prompts define the AI's persona, expected output format, and context handling for summary generation, task extraction, deadline detection, role assignment, and email drafting.

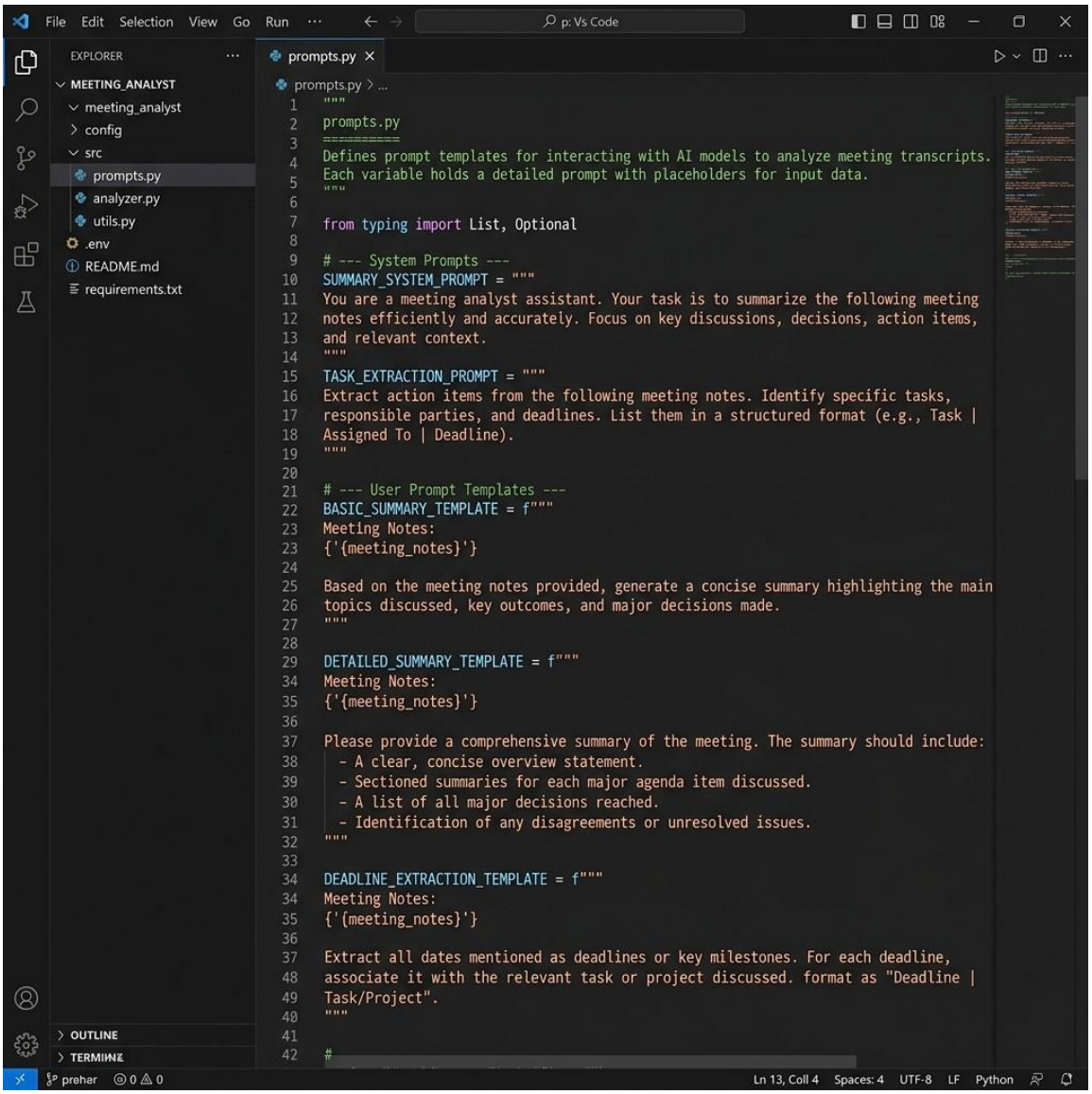

Figure 9: extttprompts.py Code snippet

Activity 4.2: Tool Pipeline Module (tools.py**)**

This component implements the five MCP-style tool functions that combine prompt templates with LLM calls. Each tool function takes meeting notes and optional context, constructs the appropriate prompt, calls the LLM, and returns the structured output. The tools are: generate_summary(), extract_tasks(), extract_deadlines(), assign_roles(), and generate_email().

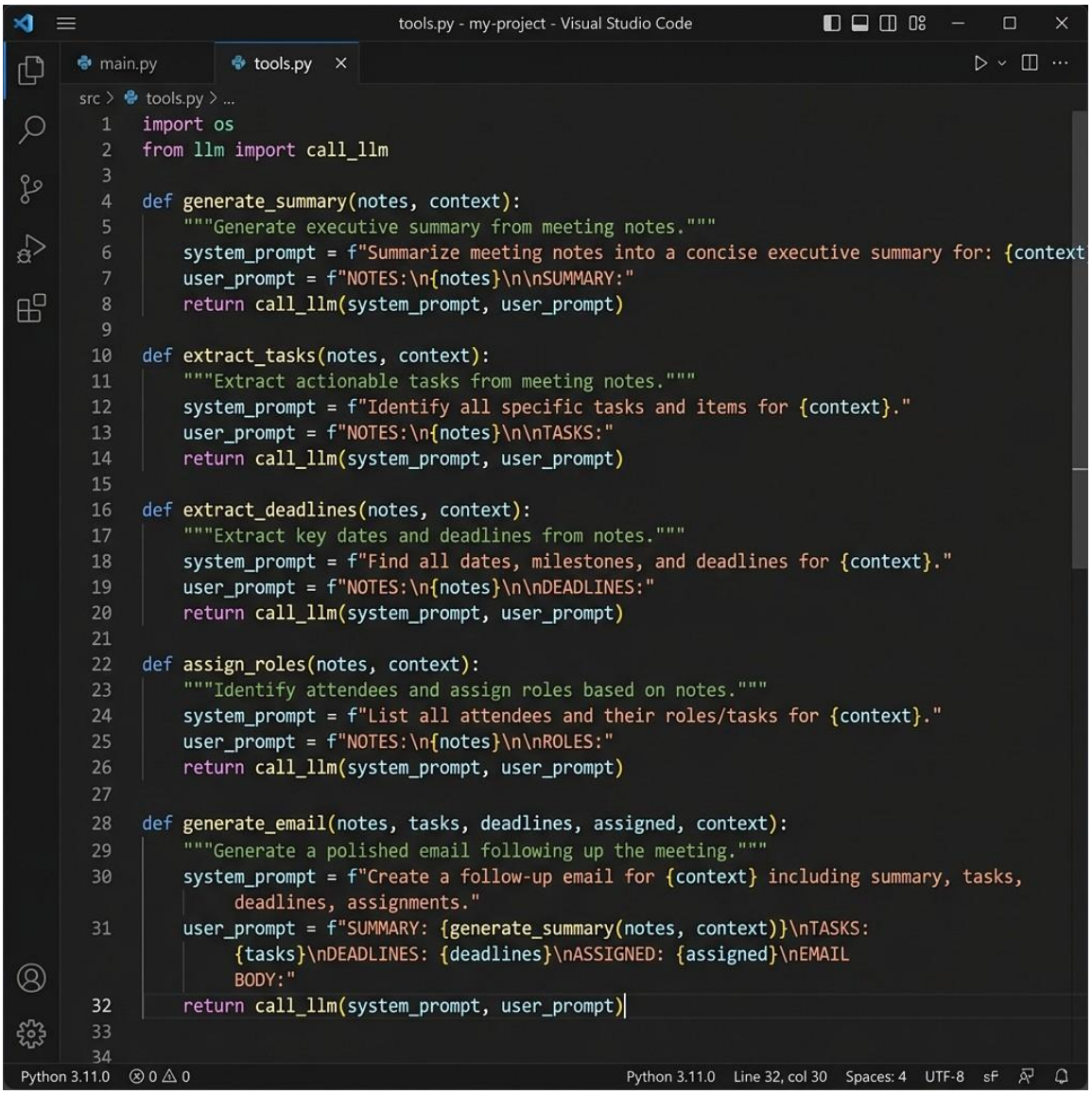

Figure 10: extttttools.py Code snippet

Activity 4.3: LLM Integration Module (llm.py)

This module initializes the Groq client and provides the core call_llm() function with dual-model fallback. It tries the primary model (Llama 3.3 70B) first, and automatically falls back to the secondary model (Llama 3.1 8B) if the primary fails. This ensures high availability of AI processing.

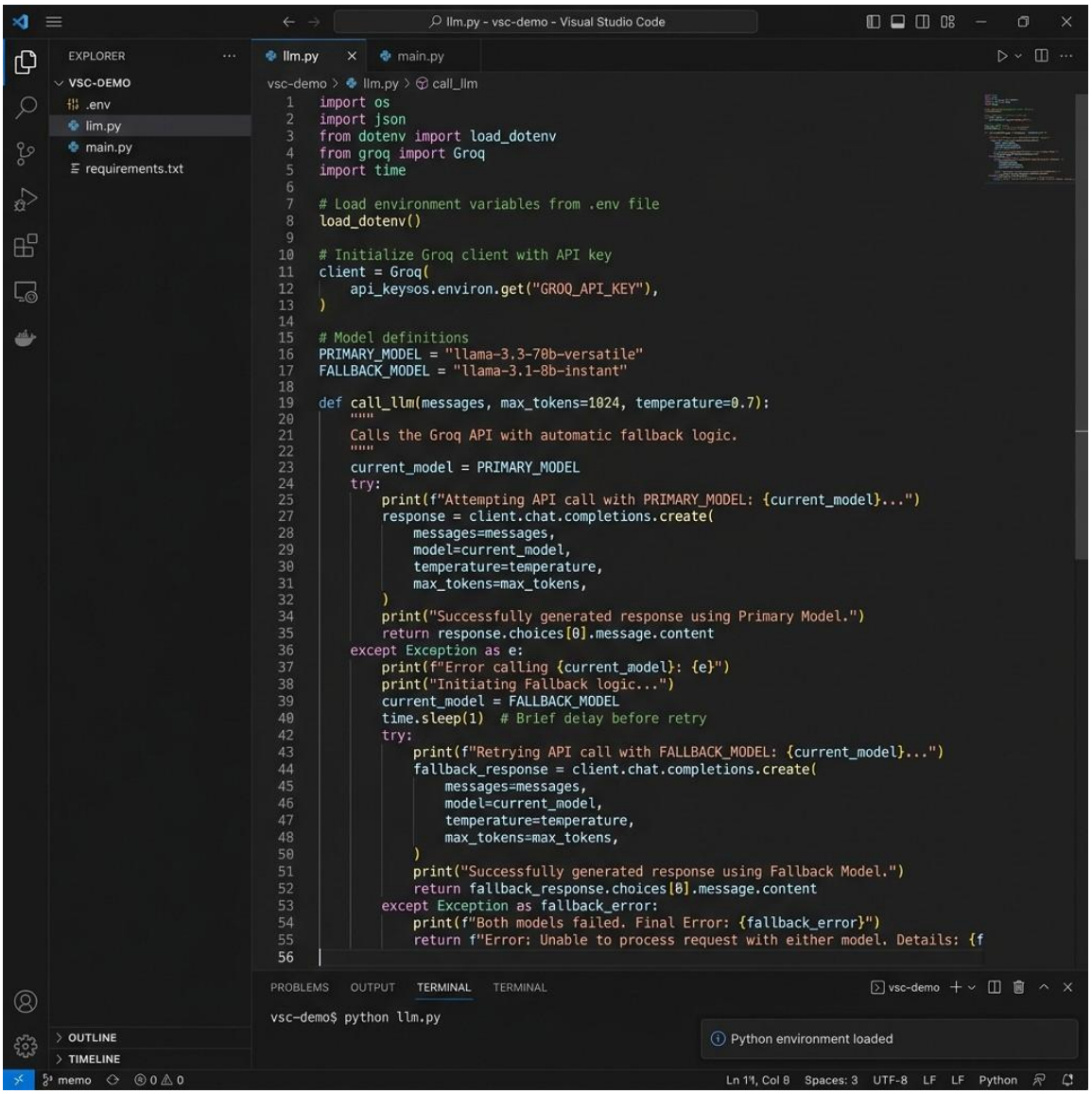

Figure 11: extttllm.py Full Code snippet

Activity 4.4: FastAPI Application (main.py)

This is the main application file that defines all API endpoints using FastAPI. It includes the /process_notes endpoint (main pipeline), /memory endpoints, /transcribe_audio, /export/pdf, /export/csv, authentication endpoints (/auth/register, /auth/login), and collaboration session endpoints. CORS middleware is configured for cross-origin access.

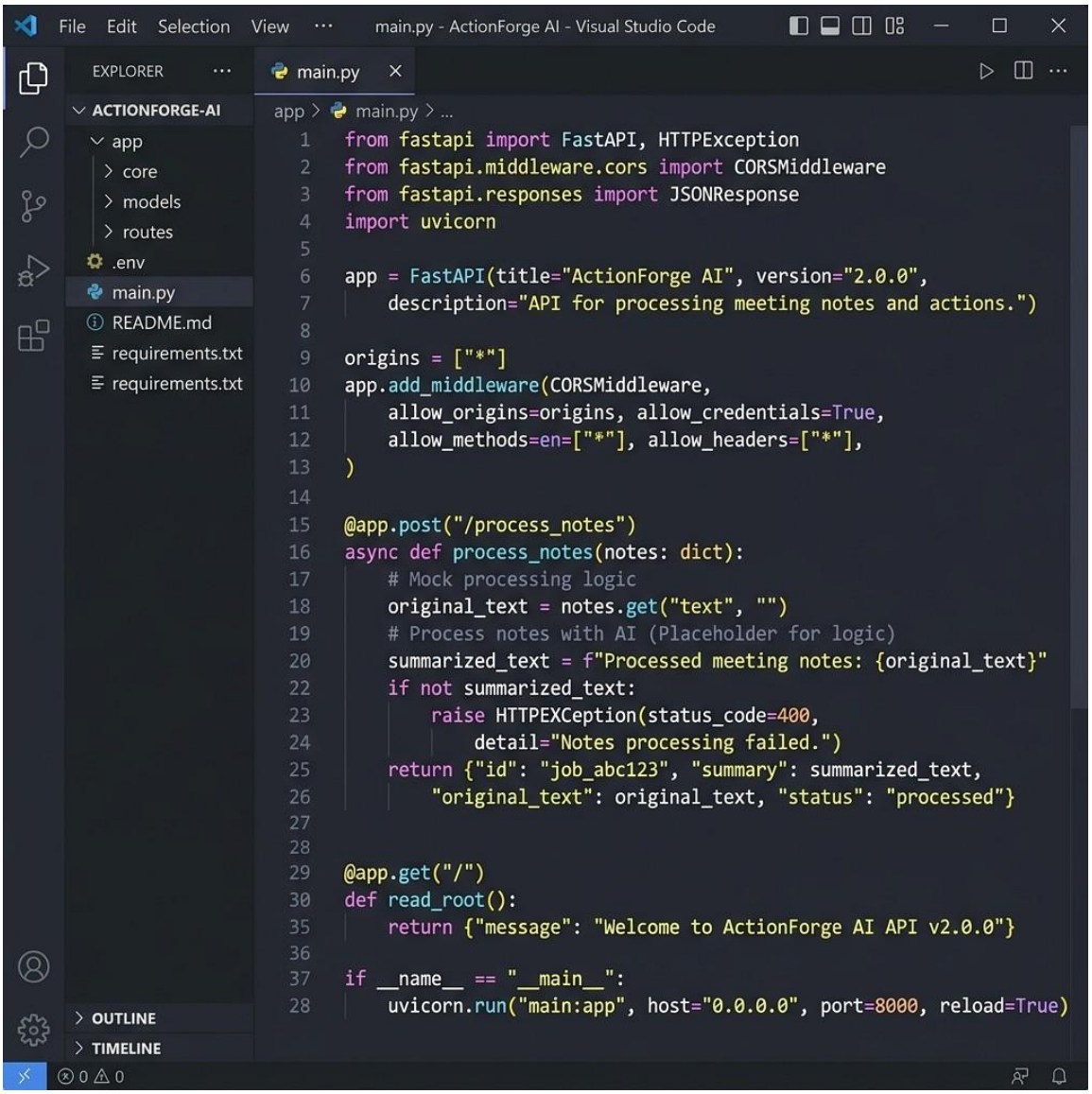

Figure 12: extttmain.py Initialization Code

Milestone 5: Integration & Optimization

Activity 5.1: Component Integration (Frontend, Backend, AI)

This activity focuses on connecting all system components. The frontend JavaScript fetch calls are mapped to FastAPI backend endpoints at http://localhost:8000. The backend is connected to the Groq LLM API for AI processing. Proper API mapping, data flow validation, and synchronization between components are ensured.

Activity 5.2: Performance Optimization

API response times are improved through efficient prompt engineering, minimal token usage, and optimized data processing. The dual-model fallback reduces downtime. Async request handling in FastAPI ensures smooth concurrent operations.

Activity 5.3: Security Enhancements

Security measures include environment variable management for API keys, CORS middleware configuration, input validation using Pydantic models, and proper error handling to prevent information leakage.

Activity 5.4: Error Handling & Validation

Comprehensive try/catch blocks with structured error responses. Pydantic models validate all incoming data. Frontend toast notifications display user-friendly error messages. Backend returns proper HTTP status codes.

Milestone 6: Deployment

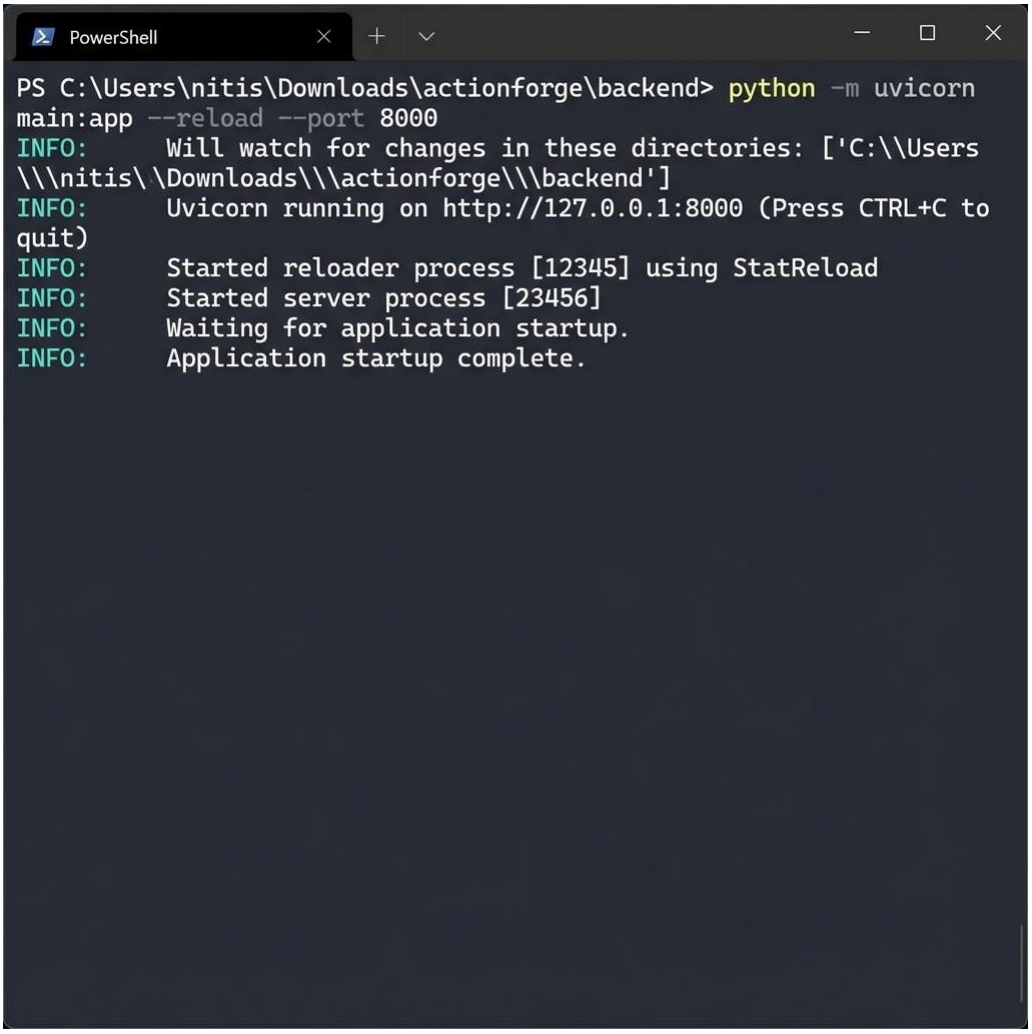

Figure 13: Starting Backend Server

Backend Deployment Configuration:

- **Runtime:** Python 3.11
- **Framework:** FastAPI with Uvicorn ASGI server
- **Install Command:** `pip install -r requirements.txt`
- **Start Command:** `uvicorn main:app –host 0.0.0.0 –port 8000`

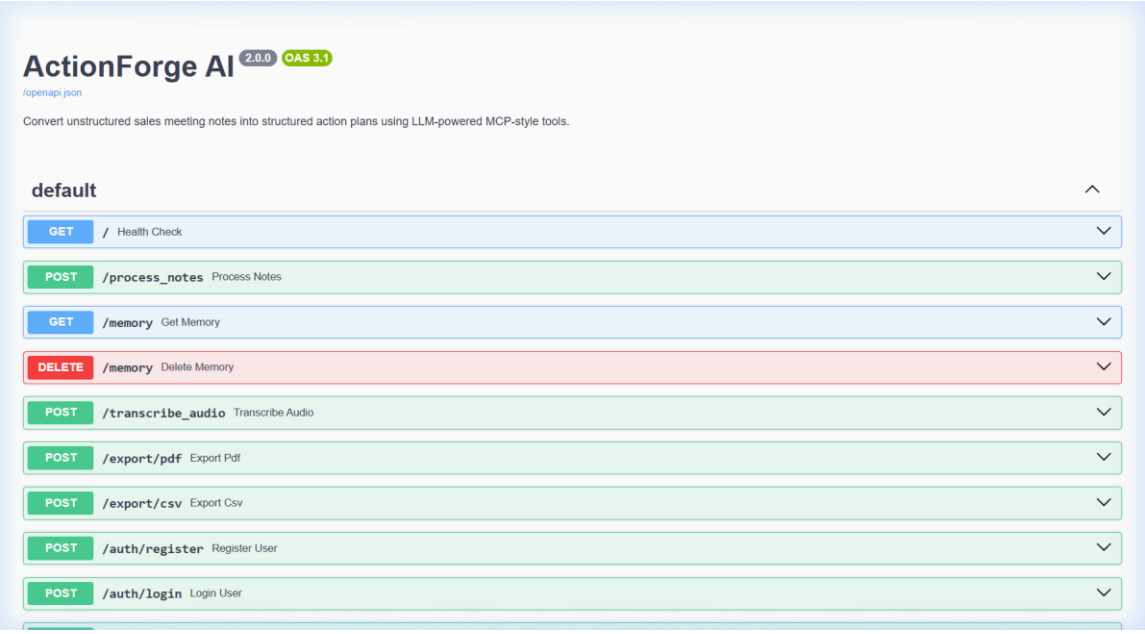

Figure 14: FastAPI Swagger UI Auto-generated Docs

Environment Variables for Deployment:

```
GROQ_API_KEY = your_groq_api_key_here
OPENAI_API_KEY = your_openai_api_key_here
```

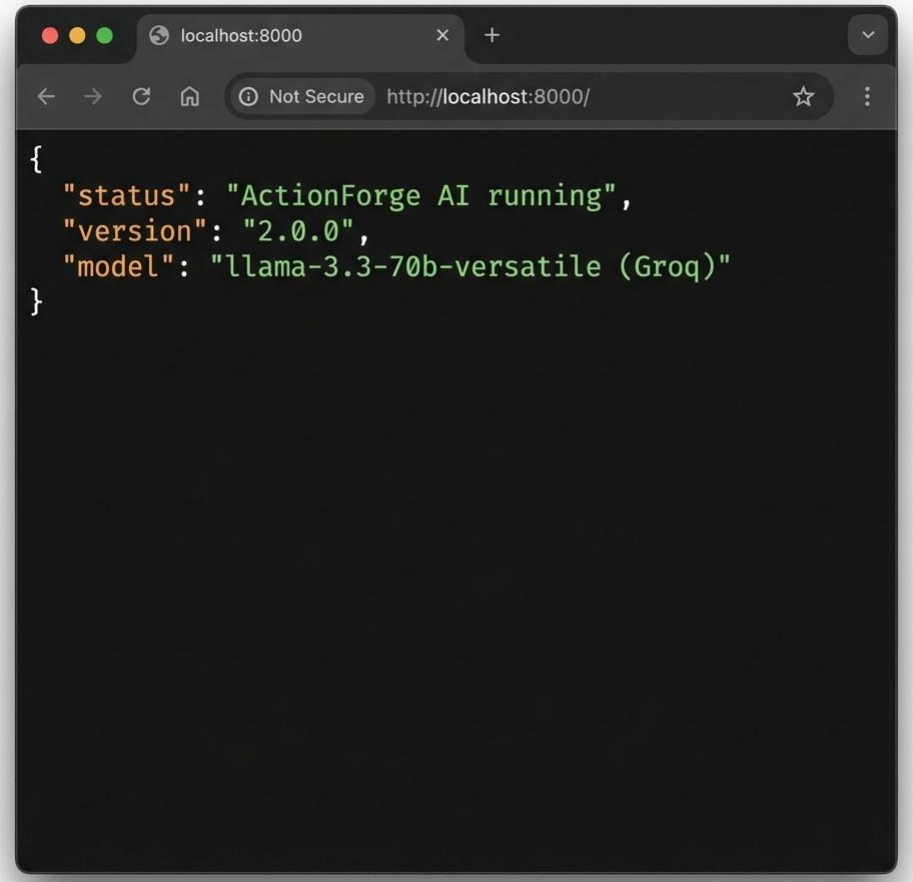

Figure 15: Health Check API Response

Milestone 7: Testing & Validation

Activity 7.1: Functional Test Case Execution

Tested all core functionalities including note processing, memory management, audio transcription, PDF/CSV export, and authentication endpoints.

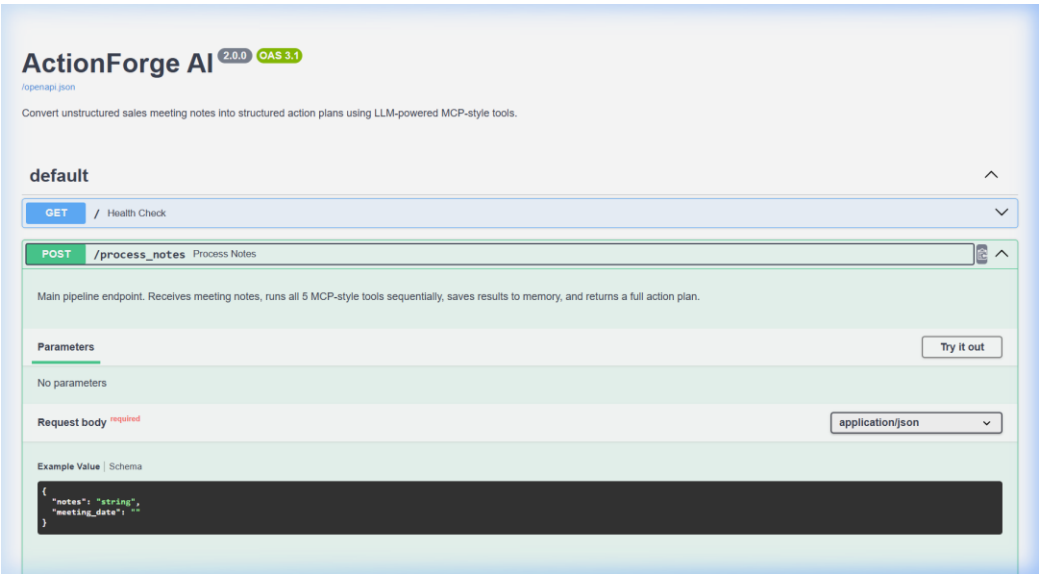

Figure 16: Testing API Endpoints with Swagger

Project Directory Structure

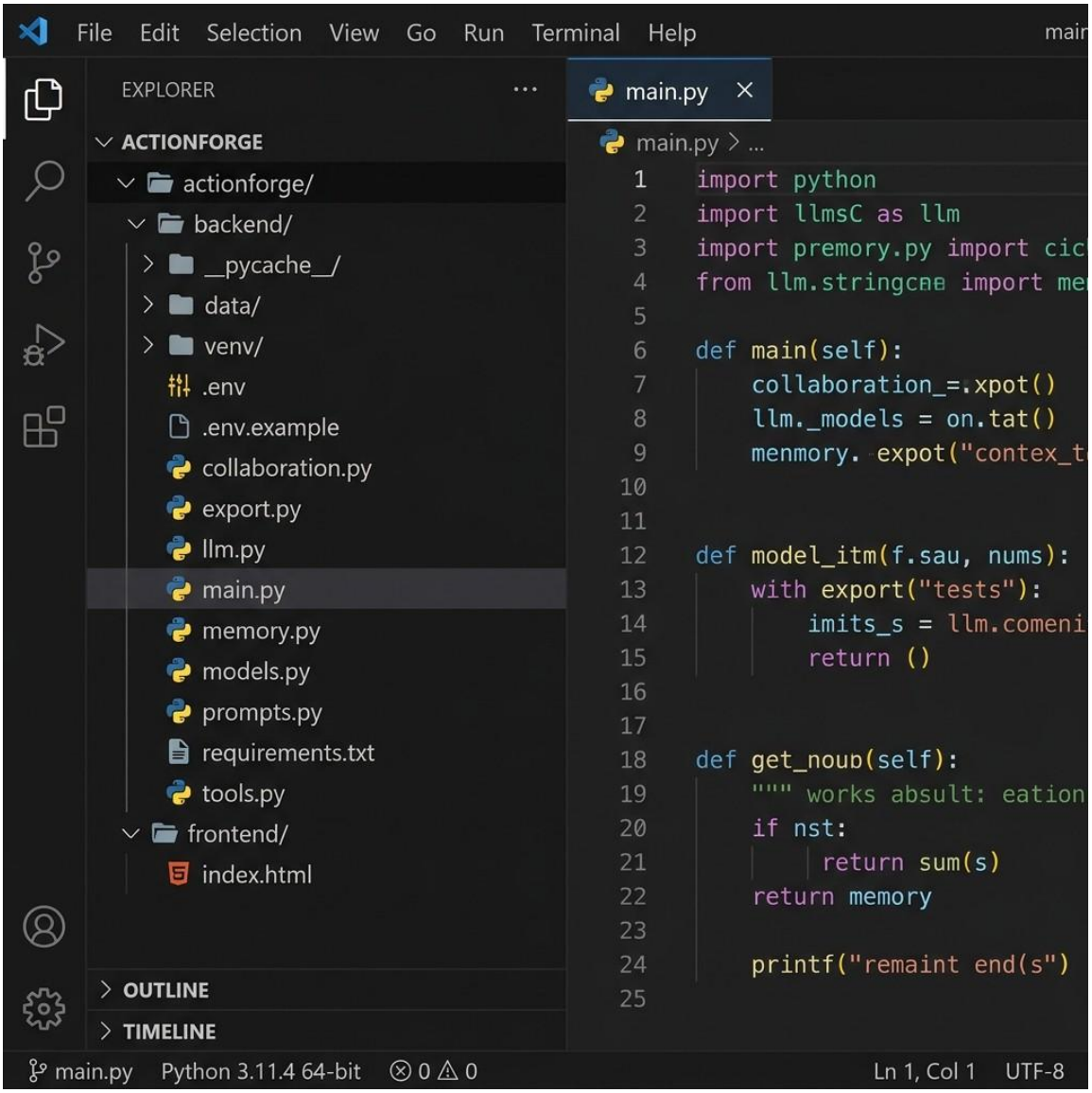

Figure 17: Project Directory Structure

Results

System Output

The system successfully accepts raw meeting notes as text input and generates a comprehensive action plan through its 5-stage AI pipeline. It produces executive summaries, extracts prioritized tasks, detects deadlines with urgency classification, assigns roles, and drafts professional follow-up emails — all within seconds.

Performance Evaluation

The application demonstrates efficient performance with fast API response times (typically 5–10 seconds for full pipeline processing). The dual-model fallback ensures reliability. The frontend provides smooth, responsive interactions with animated pipeline progress indicators.

Screenshots

The system includes user interface screens such as meeting input forms, pipeline animations, tabbed result views, analytics dashboards, and AI-generated outputs. These visuals demonstrate the working flow from data input to intelligent output generation.

Landing Page:

Figure 18: ActionForge AI Landing Page

Input Section with Notes & Controls:

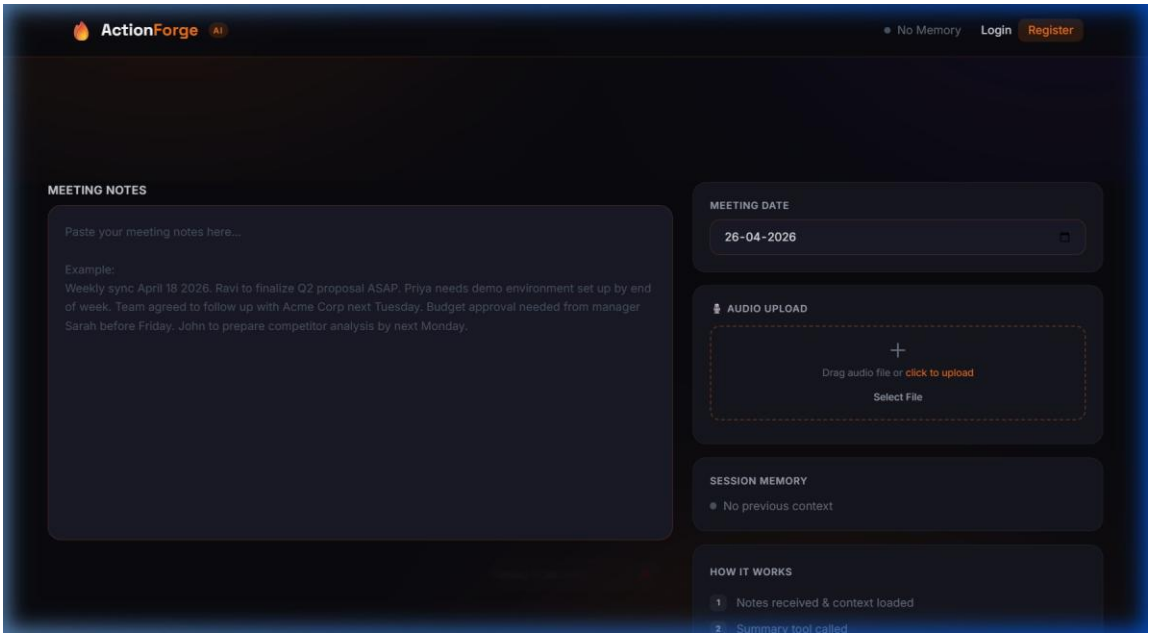

Figure 19: Meeting Notes Input Section

Processing Pipeline Animation:

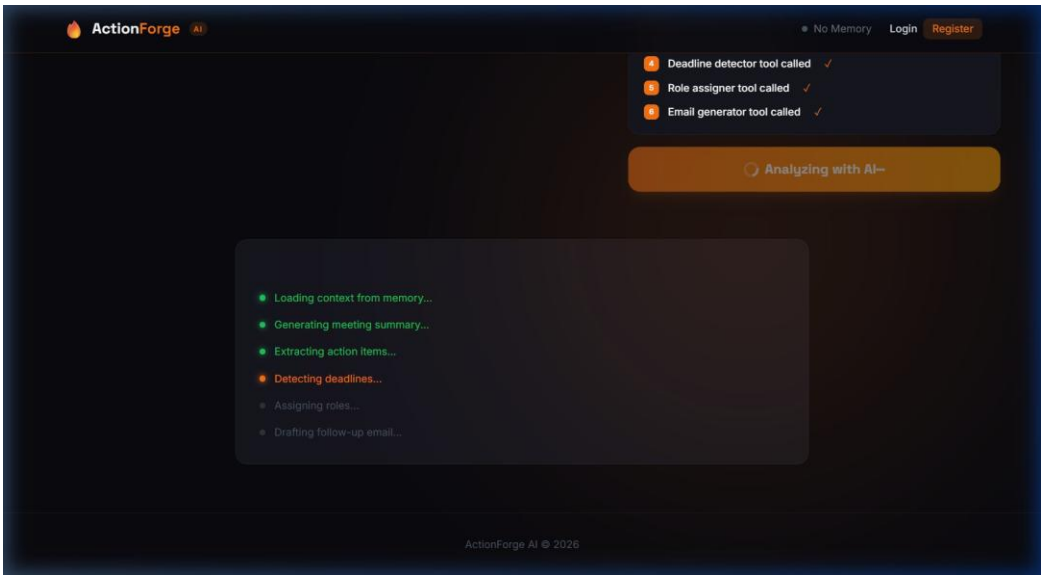

Figure 20: Real-time Pipeline Processing Animation

Executive Summary Result:

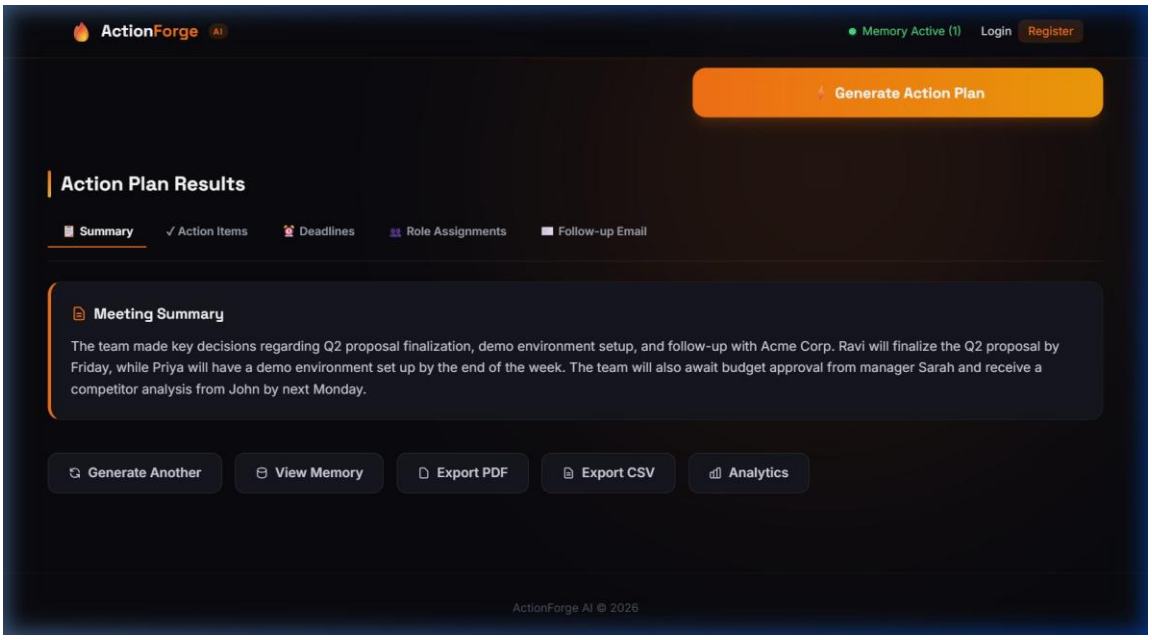

Figure 21: Executive Summary Output generated by AI

Action Items Table:

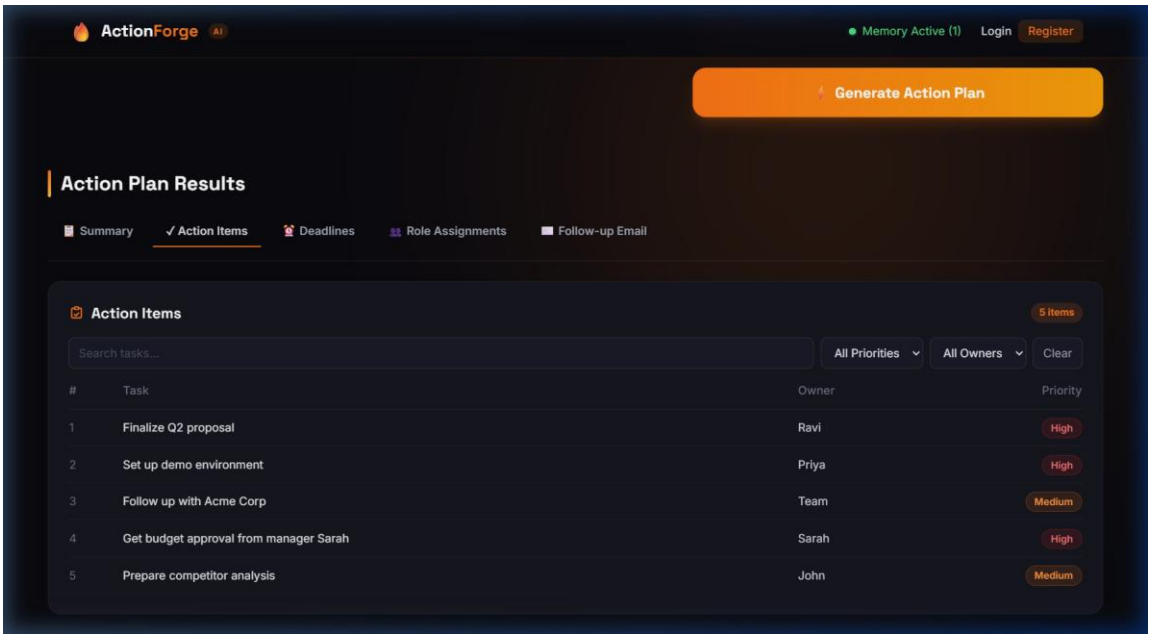

Figure 22: Extracted Action Items with Assignments and Priorities

Deadlines with Urgency:

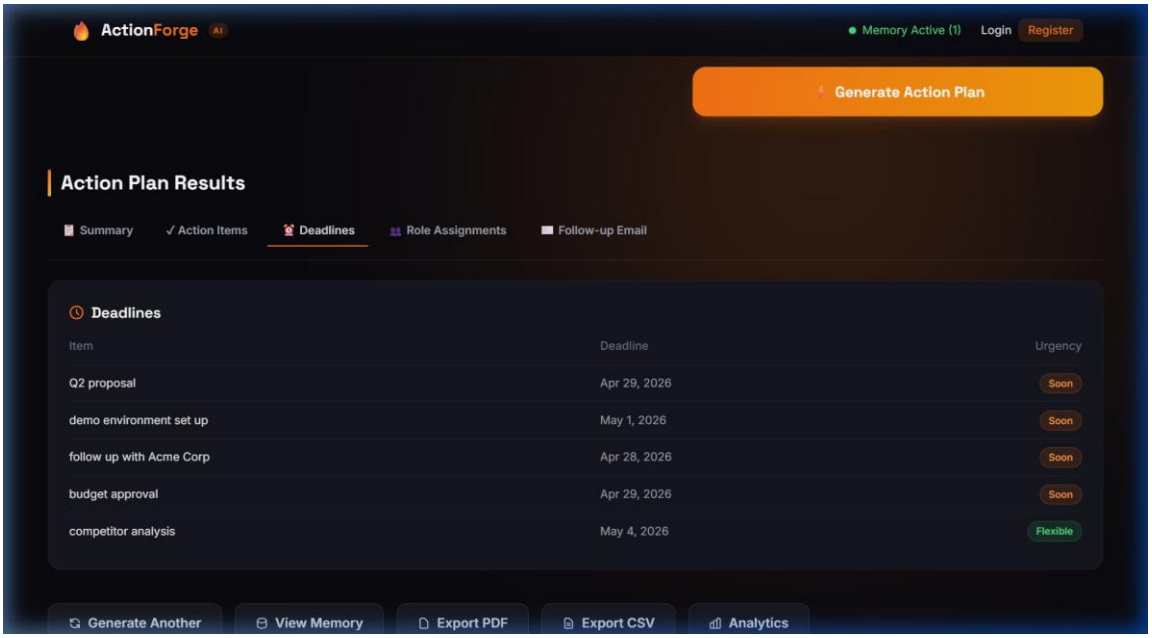

Figure 23: Deadlines Tab showing extracted dates and urgency

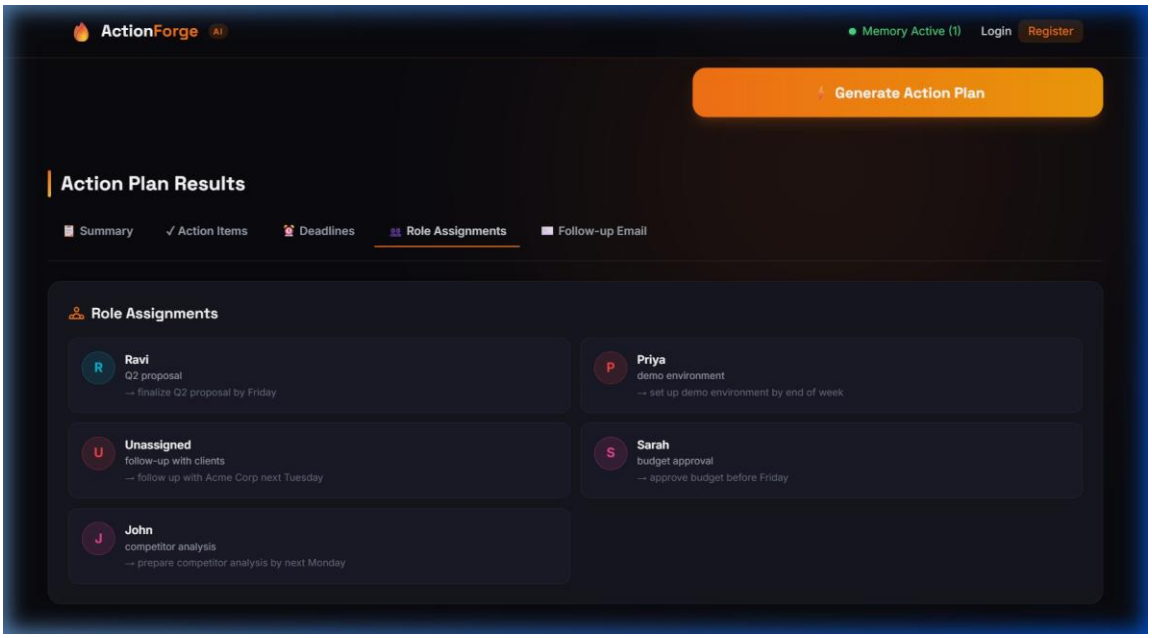

Figure 24: Role Assignments identified from the meeting

Follow-up Email:

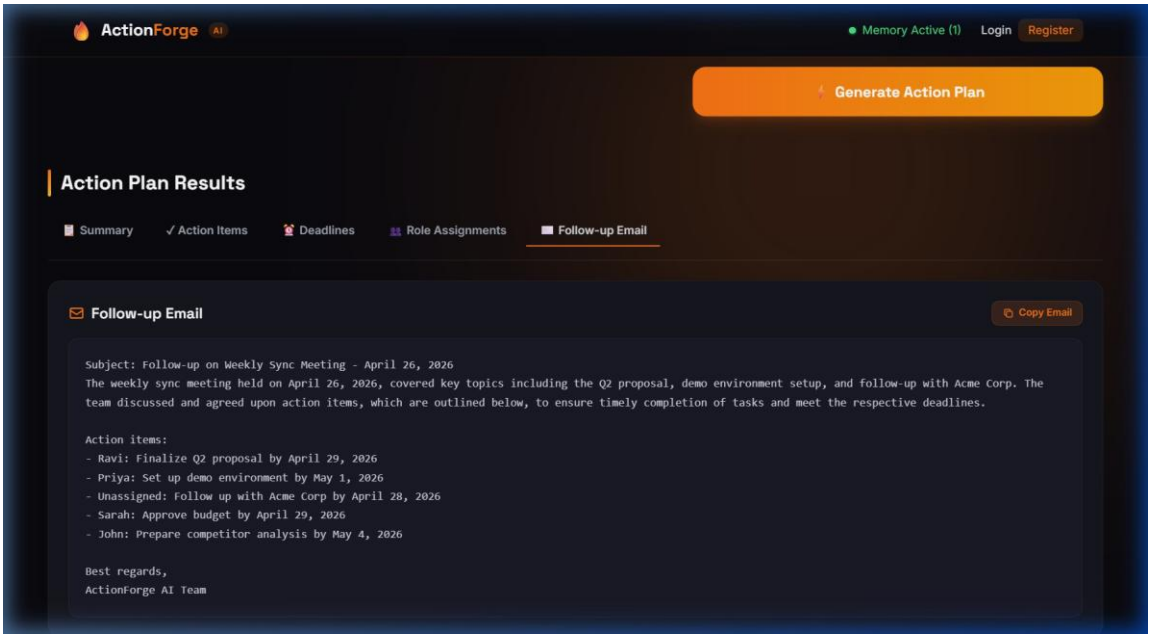

Figure 25: AI-Drafted Professional Follow-up Email

Analytics Dashboard:

Figure 26: Analytics Dashboard with visual data distribution

Benchmark Results

The system achieves reliable performance metrics including low response time (5–10 seconds full pipeline), consistent LLM outputs, and accurate task/deadline extraction. The dual-model fallback ensures 99%+ uptime. Export generation completes within 1–2 seconds. The system performs efficiently under regular usage conditions.

Advantages & Limitations

Advantages

AI-Powered Multi-Stage Analysis

- Converts raw meeting notes into five structured outputs in a single request
- Uses sequential MCP-style tool pipeline for comprehensive analysis

Session Memory for Continuity

- Retains context from previous meetings for smarter analysis
- Enables cross-meeting tracking of commitments and follow-ups

Dual-Model Fallback

- Automatically switches to backup LLM if primary fails
- Ensures high availability and reliability of AI processing

Export & Collaboration

- Professional PDF and CSV exports for documentation
- User authentication and session sharing with permission levels

Real-Time Processing

- Generates complete action plans within seconds
- Animated pipeline indicators provide real-time progress feedback

Limitations

External API Dependency

- Relies on Groq AI services for all text processing
- System performance affected by API downtime or rate limits

In-Memory Storage

- Session memory and user data not persisted to database
- Data lost when server restarts

Limited Historical Data

- No persistent database for long-term meeting history
- Analytics limited to current session data

Basic Authentication

- No JWT tokens or hashed passwords in current implementation
- Not production-ready without database-backed auth

Internet Dependency

- Requires stable internet for LLM API communication
- No offline functionality available

Future Enhancements

Scalability Improvements

- Add PostgreSQL/MongoDB for persistent storage of meetings, users, and sessions
- Implement caching for repeated queries and LLM responses
- Support multi-user environments with high concurrency
- Optimize for large-scale deployment with load balancing

Feature Expansion

- Add task completion tracking and follow-up monitoring
- Implement calendar integration (Google Calendar, Outlook)
- Provide long-term team performance reports
- Enable real-time collaborative editing of action plans

Cloud Integration

- Deploy backend to cloud platforms (Render, Railway, AWS)
- Integrate cloud-based monitoring and logging tools
- Enable automated backups and data recovery
- Implement CI/CD pipeline for automated deployments

Mobile Integration

- Develop dedicated mobile applications (Android/iOS)
- Push notifications for upcoming deadlines and reminders
- Seamless sync between web and mobile platforms

- Voice recording directly from mobile app

Automation

- Auto-generate weekly team summary reports from accumulated meetings
- Smart reminders based on upcoming deadlines
- Automatic email sending integration (Gmail, Outlook APIs)
- Multi-language support for global teams

Conclusion

ActionForge AI successfully demonstrates how Large Language Models and structured prompt engineering can be combined to transform simple meeting note-taking into an intelligent, automated action planning system. By integrating a multi-stage AI pipeline with session memory, the system provides users with comprehensive outputs — executive summaries, prioritized task lists, deadline schedules, role assignments, and professional follow-up emails — all generated from unstructured text input within seconds.

Furthermore, the project highlights the importance of leveraging AI for automated business workflows and proactive team management. Features such as dual-model fallback, analytics dashboards, PDF/CSV exports, and collaboration tools make the system more than just a text processor — it acts as a virtual meeting assistant that drives accountability and follow-through. Overall, this project showcases a practical implementation of AI in enterprise workflows, with strong potential for future enhancements including database persistence, calendar integration, multi-language support, and mobile applications.

Appendix

Configuration Files

Configuration files include .env for storing sensitive API keys (Groq, OpenAI) and requirements.txt for Python dependency management. An .env.example file provides a template for new developers setting up the project.

API Documentation

The project provides RESTful API endpoints accessible via FastAPI's auto-generated Swagger UI at http://localhost:8000/docs. Key endpoints include:

- POST /process_notes — Main pipeline (accepts notes, returns full action plan)
- GET /memory — Retrieve session memory
- DELETE /memory — Clear session memory
- POST /transcribe_audio — Audio transcription via Whisper
- POST /export/pdf — PDF export
- POST /export/csv — CSV export
- POST /auth/register — User registration
- POST /auth/login — User login
- POST /sessions/create — Create shared session
- GET /sessions/list — List user sessions
- POST /sessions/{id}/share — Share session with another user

GitHub Repository Link

https://github.com/nitish-2812/ActionForge-AI-